

Database: AdventureWorks

Table of Contents

- 1. Overview
 - 2. Scope and Prerequisites
 - 3. Key Concepts
 - 4. Pre-Migration Planning
 - 5. Migration Procedures
 - 6. Verification and Testing
 - 7. Rollback Procedures
 - 8. Troubleshooting
 - 9. Appendix - T-SQL Scripts
-

1. Overview

This SOP provides detailed instructions for migrating a Transparent Data Encryption (TDE)-protected SQL Server database to another SQL Server instance while maintaining encryption integrity and security. This procedure specifically applies to the AdventureWorks database.

Key Objectives

- Move the TDE-encrypted AdventureWorks database to a new instance without data loss
 - Preserve all encryption certificates and keys
 - Minimize downtime during migration
 - Maintain compliance and audit trails
-

2. Scope and Prerequisites

2.1 Scope

This SOP covers the complete lifecycle of migrating a TDE-protected database between SQL Server instances:

- Detaching the source database
- Backing up and transferring encryption certificates and keys
- Moving database files to the destination server
- Recreating the encryption infrastructure on the destination
- Attaching and verifying the database

2.2 Prerequisites

Source Server Requirements

- SQL Server 2016 or later (verified compatibility)
- AdventureWorks database exists and is TDE-encrypted
- Sufficient disk space for database backups
- Database Master Key password documented

Destination Server Requirements

- SQL Server 2016 or later (same or later version than source)
- Sufficient disk space matching source database size
- Database Master Key not yet created on destination
- System Administrator (SA) or equivalent permissions

Permissions Required

- CONTROL DATABASE permission on master database
 - CREATE CERTIFICATE permission on master database
 - VIEW DEFINITION permission on certificates
 - System Administrator role recommended
-

3. Key Concepts

3.1 Transparent Data Encryption (TDE)

TDE performs real-time I/O encryption and decryption of data and log files. The Database Encryption Key (DEK) is stored in the database boot record and encrypted using either a certificate or asymmetric key in the master database.

3.2 Critical Components

- **Database Master Key (DMK):** Encrypts the certificate in the master database
- **Server Certificate:** Encrypts the Database Encryption Key
- **Database Encryption Key (DEK):** Encrypts the actual database files
- **Certificate Backup Files:** .cer and .pvk files required for recreation

3.3 Important Limitations

- The certificate and private key MUST be recreated on the destination server before attaching the database
 - Private key password may differ from Database Master Key password
 - Both certificate and private key backup files must be retained for recovery
 - Database cannot be attached without the proper certificate in place
-

4. Pre-Migration Planning

4.1 Pre-Migration Checklist

- ☐ Document all Database Master Key passwords (source and destination)
- ☐ Verify certificate name and SUBJECT on source server
- ☐ Create full backup of AdventureWorks database
- ☐ Identify all database files (MDF, LDF, NDF if present)
- ☐ Verify disk space on destination server
- ☐ Schedule maintenance window (minimum 2-4 hours)
- ☐ Notify all stakeholders and dependent applications
- ☐ Create detailed rollback plan
- ☐ Have test restore procedure validated

4.2 Data Collection

Source SQL Server:

- Version: _____
- Edition: _____
- Database size: _____
- Certificate name: _____
- Database file paths: _____

Destination SQL Server:

- Version: _____
- Edition: _____
- Available disk space: _____
- Target database file paths: _____

5. Migration Procedures

5.1 Step 1: Verify TDE Status on Source Server

Execute the following query on the source server to confirm TDE is enabled:

```
USE AdventureWorks;  
GO
```

```
SELECT name, is_encrypted  
FROM sys.databases  
WHERE name = 'AdventureWorks';  
GO
```

Expected result: is_encrypted = 1 (TRUE)

5.2 Step 2: Identify and Document Certificate and Keys

Execute the following query on the source server:

```
USE master;  
GO
```

```
SELECT c.name, c.subject, c.expiry_date  
FROM sys.certificates c  
WHERE c.name NOT LIKE '###%';  
GO
```

Document the certificate name (e.g., "TestSQLServerCert") for use in later steps.

5.3 Step 3: Back Up the Certificate and Private Key

On the source SQL Server, execute the following T-SQL to back up the certificate and private key. **This step is CRITICAL** and must be completed before detaching the database.

```
USE master;  
GO
```

```
BACKUP CERTIFICATE TestSQLServerCert  
TO FILE = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestSQLServerCert.cer'  
WITH PRIVATE KEY (  
    FILE = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SQLPrivateKeyFile.pvk',  
    ENCRYPTION BY PASSWORD = '<SecurePassword>');  
GO
```

IMPORTANT: Store the password used in encryption in a secure location (password manager, secure vault). You will need this password on the destination server.

5.4 Step 4: Copy Certificate and Key Files to Destination

Using Windows Explorer or file copy commands, copy the following files from the source server to the destination server at the same path:

- TestSQLServerCert.cer
- SQLPrivateKeyFile.pvk

PowerShell example:

```
Copy-Item -Path "C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestSQLServerCert.cer" `
-Destination "\\DestinationServer\C$\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\" -Force
```

```
Copy-Item -Path "C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SQLPrivateKeyFile.pvk" `
-Destination "\\DestinationServer\C$\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\" -Force
```

Verify file permissions are consistent with other SQL Server files.

5.5 Step 5: Kill All Active Connections and Detach Database

Execute the following T-SQL on the source server to disconnect all users and detach the database:

```
USE master;
GO
```

```
ALTER DATABASE AdventureWorks SET OFFLINE WITH ROLLBACK IMMEDIATE;
GO
```

```
EXEC sp_detach_db @dbname = N'AdventureWorks';
GO
```

Verify the detach is successful. You should see a message: "Msg 5178, Level 16, State 1... Database detached successfully."

5.6 Step 6: Copy Database Files to Destination

Using file copy tools (Robocopy recommended for large files), copy the AdventureWorks database files from source to destination at the same path:

Required files: - AdventureWorks.mdf (primary data file) - AdventureWorks_log.ldf (transaction log) - Any .ndf files (secondary data files, if present)

Robocopy example:

```
robocopy "\\SourceServer\C$\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\" ^
"C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\" ^
AdventureWorks.mdf AdventureWorks_log.ldf /MT:32 /Z /R:3 /W:10
```

Note: Use /MT:32 for multi-threaded copy for larger files. /Z enables restartable mode for large transfers.

5.7 Step 7: Create Database Master Key on Destination

On the destination SQL Server, execute the following T-SQL to create the Database Master Key. Use the same password or a new one (recommended: use a new, strong password):

```
USE master;
GO
```

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<StrongPassword>';
GO
```

IMPORTANT: Store this password securely. It is different from the private key password used in Step 3.

5.8 Step 8: Restore Certificate and Private Key on Destination

On the destination SQL Server, execute the following T-SQL to recreate the certificate using the backup files:

```
USE master;
```

```
GO
```

```
CREATE CERTIFICATE TestSQLServerCert
```

```
FROM FILE = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestSQLServerCert.cer'
```

```
WITH PRIVATE KEY (
```

```
FILE = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SQLPrivateKeyFile.pvk',
```

```
DECRYPTION BY PASSWORD = '<PrivateKeyPassword>');
```

```
GO
```

NOTE: Use the **EXACT password** from Step 3 for the DECRYPTION BY PASSWORD clause. This is the password you used when backing up the certificate.

5.9 Step 9: Attach Database on Destination

On the destination SQL Server, execute the following T-SQL to attach the AdventureWorks database:

```
USE master;
```

```
GO
```

```
CREATE DATABASE AdventureWorks
```

```
ON (FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\AdventureWorks.mdf'),
```

```
(FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\AdventureWorks_log.LDF')
```

```
FOR ATTACH;
```

```
GO
```

If .ndf files exist, add them to the CREATE DATABASE statement:

```
CREATE DATABASE AdventureWorks
```

```
ON (FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\AdventureWorks.mdf'),
```

```
(FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\AdventureWorks_log.LDF'),
```

```
(FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\AdventureWorks_2.ndf')
```

```
FOR ATTACH;
```

```
GO
```

Success message: "Database 'AdventureWorks' created successfully."

6. Verification and Testing

6.1 Verify TDE Status

Execute the following on the destination server to confirm TDE is active:

```
USE AdventureWorks;
```

```
GO
```

```
SELECT name, is_encrypted
```

```
FROM sys.databases
```

```
WHERE name = 'AdventureWorks';
```

```
GO
```

Expected result: is_encrypted = 1

6.2 Verify Certificate

Execute the following to confirm the certificate exists and is valid:

```
USE master;
```

```
GO
```

```
SELECT name, subject, expiry_date
```

```
FROM sys.certificates
```

```
WHERE name = 'TestSQLServerCert';
```

```
GO
```

Expected result: Certificate name, subject, and future expiry date

6.3 Verify Database Encryption Key

Execute the following to confirm the DEK is functional:

```
USE AdventureWorks;
```

```
GO
```

```
SELECT database_id, encryption_state, percent_complete
```

```
FROM sys.dm_database_encryption_keys;
```

```
GO
```

Expected result: encryption_state = 3 (encrypted), percent_complete = 0 (complete)

6.4 Test Database Functionality

Execute basic read operations to ensure data is accessible:

```
USE AdventureWorks;
```

```
GO
```

```
SELECT COUNT(*) FROM Sales.SalesOrderHeader;
```

```
GO
```

```
SELECT TOP 5 * FROM Production.Product;
```

```
GO
```

Expected result: Normal query execution with data returned

6.5 Verify Backup and Recovery

Test creating and restoring a backup of the migrated database:

```
USE master;
```

```
GO
```

```
BACKUP DATABASE AdventureWorks
```

```
TO DISK = 'C:\Backups\AdventureWorks_Full.bak'
```

```
WITH FORMAT, INIT, STATS = 10;
```

```
GO
```

Expected result: Successful backup completion

7. Rollback Procedures

If errors occur during migration, follow these steps to restore the original state:

7.1 If Database Fails to Attach

1. Check SQL Server error log for detailed error messages:

```
EXEC xp_readerrorlog;  
GO
```

2. Common attachment failures and solutions:

Error	Cause	Solution
Cannot find certificate	Certificate not recreated properly	Verify certificate files exist and password is correct
File not found	Database file paths incorrect	Verify MDF/LDF file locations match the CREATE DATABASE statement
Cannot open database	Database Master Key not created	Execute CREATE MASTER KEY command on destination

7.2 If Certificate Recreation Fails

1. Verify certificate backup files exist:

```
dir "C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestSQL*"
```

2. Verify password is correct (case-sensitive)
3. Re-copy certificate files from source server if files are corrupted

7.3 Rollback to Source Server

If migration cannot be completed:

1. On source server, bring database back online:

```
USE master;  
GO
```

```
ALTER DATABASE AdventureWorks SET ONLINE;  
GO
```

2. Verify database is accessible and TDE is still enabled

8. Troubleshooting

8.1 Common Issues and Resolutions

Issue	Cause	Resolution
Cannot decrypt certificate	Incorrect password	Verify password is case-sensitive and matches backup password
File not found error	Database files not copied to destination	Copy files again using Robocopy; verify paths match exactly
Database in suspect mode	Encryption key corruption	Restore from backup on destination; verify certificate is correct
Slow attachment/startup	Network latency or disk I/O	Use local fast storage; monitor disk and network during attach

8.2 Checking SQL Server Error Logs

When troubleshooting, always check the SQL Server error log:

```
EXEC xp_readerrorlog 0, 1, NULL, NULL, '-24 hours';
GO
```

This command retrieves all error log entries from the last 24 hours.

8.3 Testing Certificate Availability

Verify the certificate can decrypt the DEK:

```
USE AdventureWorks;
GO

SELECT DB_NAME(database_id) AS Database_Name,
       encryption_state,
       encryptor_thumbprint
FROM sys.dm_database_encryption_keys;
GO
```

9. Appendix - T-SQL Scripts

9.1 Complete Source Server Script

Execute this complete script on the SOURCE server:

```
-- =====
-- SOURCE SERVER: TDE Protected Database Migration - Steps 1-5
-- =====

-- STEP 1: Verify TDE Status
USE AdventureWorks;
GO

SELECT name, is_encrypted
FROM sys.databases
WHERE name = 'AdventureWorks';
GO

-- Expected: is_encrypted = 1 (TRUE)

-- =====
-- STEP 2: List Certificates
-- =====

USE master;
GO

SELECT name, subject, expiry_date, thumbprint
FROM sys.certificates
WHERE name NOT LIKE '##%';
GO
```



```

-- =====
-- STEP 3: Backup Certificate and Private Key
-- CRITICAL: Adjust password and paths as needed
-- =====

BACKUP CERTIFICATE TestSQLServerCert
  TO FILE = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestSQLServerCert.cer'
  WITH PRIVATE KEY (
    FILE = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SQLPrivateKeyFile.pvk',
    ENCRYPTION BY PASSWORD = '<SecurePassword>');

```

```

-- =====
-- STEP 5: Detach Database
-- =====

```

```

USE master;
GO

```

```

ALTER DATABASE AdventureWorks SET OFFLINE WITH ROLLBACK IMMEDIATE;
GO

```

```

EXEC sp_detach_db @dbname = N'AdventureWorks';

```

9.2 Complete Destination Server Script

Execute this complete script on the DESTINATION server:

```

-- =====
-- DESTINATION SERVER: TDE Protected Database Migration - Steps 7-9
-- =====

```

```

-- STEP 7: Create Database Master Key on Destination

```

```

USE master;
GO

```

```

CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<StrongPassword>';
GO

```

```

-- =====
-- STEP 8: Recreate Certificate from Backup Files
-- CRITICAL: Use the EXACT password from source backup
-- =====

```

```

CREATE CERTIFICATE TestSQLServerCert
  FROM FILE = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestSQLServerCert.cer'
  WITH PRIVATE KEY (
    FILE = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SQLPrivateKeyFile.pvk',
    DECRYPTION BY PASSWORD = '<PrivateKeyPassword>');

```

```

-- =====
-- STEP 9: Attach Database
-- =====

```

```

CREATE DATABASE AdventureWorks
  ON (FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\AdventureWorks.mdf'),
  (FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\AdventureWorks_log.LDF')
  FOR ATTACH;

```

-- =====
-- POST-ATTACHMENT VERIFICATION
-- =====

USE AdventureWorks;
GO

SELECT name, is_encrypted
FROM sys.databases
WHERE name = 'AdventureWorks';
GO

SELECT COUNT(*) AS [Row Count]
FROM Sales.SalesOrderHeader;
GO

Document History

Version	Date	Author	Changes
1.0	2026-07-21	DBA Team	Initial document creation

End of Document

Reference:

<https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/move-a-tde-protected-database-to-another-sql-server?view=sql-server-ver17>